

Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Interação em Ambientes Virtuais

Projeto final

Turma: 61D

Nome: João Ramos Número: 50730

Nome: Fábio Pestana Número: 50756

Professor: Diogo Cabral

Data: 28 de junho de 2025

Índice

Lista de Figuras 3
Introdução 4
Objetivo do Projeto 5
Geração Procedimental 6
Aprendizagem Automática 10
Objetivo do Treino 10
Paralelismo no Treino 10
Algoritmo Utilizado 11
Parâmetros de treino para os comportamentos: *Predator* e *Prey* 11
Comportamento dos Agentes 12
Scene dos Agentes 13
Scene do utilizador contra os predadores 14
Interação Pessoa-Máquina 15
Conclusão 17
Bibliografia 18

Lista de Figuras

Figura 1 - *Texture Atlas* utilizado 6
Figura 2 - Exemplo de um *chunk* no Minecraft 7
Figura 3 - Teste com terrenos menos suaves (*smooth* = 0.005f) 7
Figura 4 - Demonstração dos minérios numa gruta 8
Figura 5 - Demonstração de terrenos 8
Figura 6 - *Hotbar* utilizada no projeto 9
Figura 7 - Demonstração da troca do bloco selecionado 9
Figura 8 - Exemplo de construção por parte do jogador 9
Figura 9 - Ambiente de treino dos agentes: predadores (verde) e presas (vermelho) 10
Figura 10 - Utilizador no início da scene na "varanda" 13
Figura 11 - Utilizador a ver as simulações dos agentes 13
Figura 12 - Utilizador no início da scene (utilizador contra predadores) 14
Figura 13 - Utilizador contra dois predadores 14
Figura 14 - Utilizador contra três predadores 15
Figura 15 - Interface realizada no TouchOC 16

Introdução

Este projeto tem como objetivo a integração dos conhecimentos adquiridos ao longo da unidade curricular de Interação em Ambientes Virtuais, através do desenvolvimento de uma aplicação em Unity que combina três grandes áreas abordadas durante o semestre: Geração Procedimental, Aprendizagem Automática e Interação Pessoa-Máquina.

A proposta consiste na criação de um mundo voxel em estilo Minecraft, gerado procedimentalmente, onde agentes predadores, ensinados por algoritmos de aprendizagem por reforço, têm como missão caçar agentes presas, que têm de fugir. Os comportamentos dos agentes foram treinados com recurso a técnicas de Machine Learning.

Como forma de explorar a componente de Interação Pessoa-Máquina, foi desenvolvida uma interface de controlo para dispositivos móveis através da aplicação TouchOSC. Através desta interface, o utilizador consegue controlar o personagem jogável no computador, ao utilizar botões e controlos analógicos no ecrã do telemóvel.

Este projeto culmina os três trabalhos realizados ao longo do semestre: o primeiro focado na criação de um mundo voxel estilo Minecraft, o segundo na implementação de agentes inteligentes com aprendizagem por reforço, e o terceiro na integração de interfaces interativas com o utilizador. Ao combinar estas componentes num único projeto, foi possível consolidar e aplicar de forma prática os conhecimentos adquiridos nesta unidade curricular. Os fundamentos teóricos e detalhes dos tópicos abordados neste projeto estão documentados nos relatórios dos três trabalhos práticos anteriores, que podem ser consultados como apoio a este relatório.

Objetivo do Projeto

O principal objetivo deste projeto é integrar, de forma coesa, os três grandes tópicos abordados ao longo do semestre: Geração Procedimental, Aprendizagem Automática e Interação Pessoa-Máquina através do desenvolvimento de um jogo estilo *sandbox voxel*, inspirado em Minecraft, com apoio do motor de jogo Unity.

Na componente de **Geração Procedimental**, foi implementado um sistema de criação de mundos infinitos, dividido em *chunks* de 16×16×16 blocos. O terreno apresenta variação de altura e é composto por diferentes tipos de blocos, incluindo terra, pedra e minérios, permitindo ao jogador explorar e modificar o ambiente. O jogador pode interagir com o mundo ao destruir blocos ou colocar até oito tipos diferentes de blocos, selecionáveis através de uma *hotbar*.

Para a **Aprendizagem Automática**, o projeto conta com a implementação de agentes inteligentes que utilizam a *framework* ML-Agents do Unity. O cenário consiste num conjunto de dois ou três agentes predadores, treinados com aprendizagem por reforço, cuja tarefa é perseguir e colidir com um agente presa, que tenta fugir. O treino foi realizado num ambiente separado e recorreu à técnica de paralelismo com múltiplos ambientes simultâneos, o que permitiu uma aprendizagem mais rápida e eficiente através da acumulação acelerada de experiências.

A componente de **Interação Pessoa-Máquina** foi explorada através da criação de uma interface personalizada para telemóvel, desenvolvida com a aplicação TouchOSC. Esta interface funciona como um comando para o jogo no computador que permite controlar a personagem do jogador de forma intuitiva. A interface inclui dois analógicos, um para movimentação e outro para controlo da câmara, e vários botões que possibilitam ações como correr, saltar, sair, mudar de *scene* do Unity, selecionar blocos na *hotbar*, e interagir com o ambiente (colocar e destruir blocos).

Este projeto visa demonstrar a integração prática e funcional dos conceitos aprendidos na unidade curricular, resultando numa experiência interativa que combina geração dinâmica de mundo, inteligência artificial adaptativa e controlo remoto por interface personalizada.

Geração Procedimental

Visto que o ambiente a ser desenvolvido é representado em Voxels, o mundo criado será composto por diversos blocos representados na implementação pela classe `Block`. Cada tipo de bloco tem texturas associadas a si. Para conter as diversas texturas utilizadas nos blocos temos um *texture atlas*, que é uma imagem que contém em si múltiplas pequenas imagens, cada uma destas sendo uma textura de um lado de um bloco. Utiliza-se esta ferramenta para se tornar mais eficiente o armazenamento de texturas utilizadas no jogo, reduzindo assim o alocamento de memória e sobrecarga de *input*/*output processing system*. De seguida encontra-se o *textures atlas* utilizado.

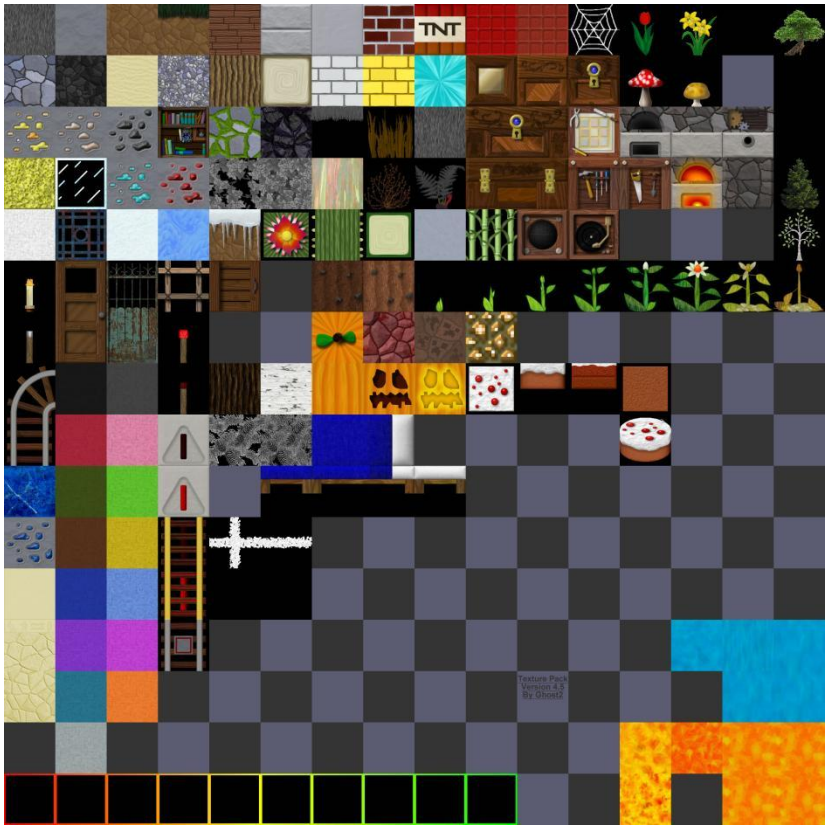


Figura 1 - *Texture Atlas* utilizado

Dado que a geração e renderização de milhares de blocos seria um processo extremamente pesado, o sistema de *chunks* (grupo de 16x16x16 blocos) é implementado de modo a otimizar este processo, visto que os *chunks* permitem agrupar blocos numa única malha, o que reduz chamadas do *draw* (processo de renderização). Para a implementação deste sistema de *chunks* criou-se a classe `Chunk` que representa as partes do mundo.



Figura 2 - Exemplo de um *chunk* no Minecraft

Para gerar o terreno, implementou-se a classe `Utils` que tem como objetivo, através da utilização do *Perlin Noise*, gerar alturas e variações naturais no terreno de forma procedimental. Esta abordagem permite criar um terreno que parece orgânico, com colinas, montanhas e outras características. A classe `Utils` inclui parâmetros que influenciam diretamente o aspeto final do terreno. O parâmetro *smooth* determina a suavidade geral e controla o quão abruptas ou planas são as variações de altura. O valor de *octaves* define quantas camadas de ruído são combinadas, sendo que cada camada adicional acrescenta mais detalhes à forma do relevo. A *persistence* regula a influência de cada camada e diminui progressivamente a amplitude do ruído de forma a equilibrar a complexidade do terreno. Já o *offset* introduz um desvio fixo nos cálculos para evitar padrões repetitivos e garantir que cada porção gerada do mundo mantenha-se visualmente distinta.



Figura 3 - Teste com terrenos menos suaves (*smooth* = 0.005f)

Por fim a classe `World` gera o mundo com a responsabilidade da criação e manutenção dinâmica do mundo voxel em torno do jogador. Atua como núcleo de controlo do ambiente, pois é esta classe que controla a forma como os blocos são organizados, atualizados e apresentados à medida que o jogador desloca-se pelo mundo. Esta estrutura garante que apenas as partes relevantes do mundo permanecem ativas em cada momento, o que permite uma

otimização do desempenho e assegura uma experiência sem queda de *frames*.
Aqui temos algumas imagens do resultado obtido na geração do mundo voxel:



Figura 4 - Demonstração dos minérios numa gruta



Figura 5 - Demonstração de terrenos

Com o mundo já pronto, queremos proporcionar ao jogador a possibilidade de interagir com o mundo que o rodeia, para tal são criadas as classes `BlockInteraction` e `HandAndBuildBlock`. A classe `BlockInteraction` tem como principal objetivo gerenciar a interação do jogador como os blocos que compõe o mundo, o que permite a destruição de blocos existentes e a construção de outros blocos onde o jogador quiser. Já a classe `HandAndBuildBlock` tem como principal objetivo permitir que o jogador alterne entre os diferentes blocos que pode construir no mundo que estão presentes numa *hotbar*.



Figura 6 - *Hotbar* utilizada no projeto

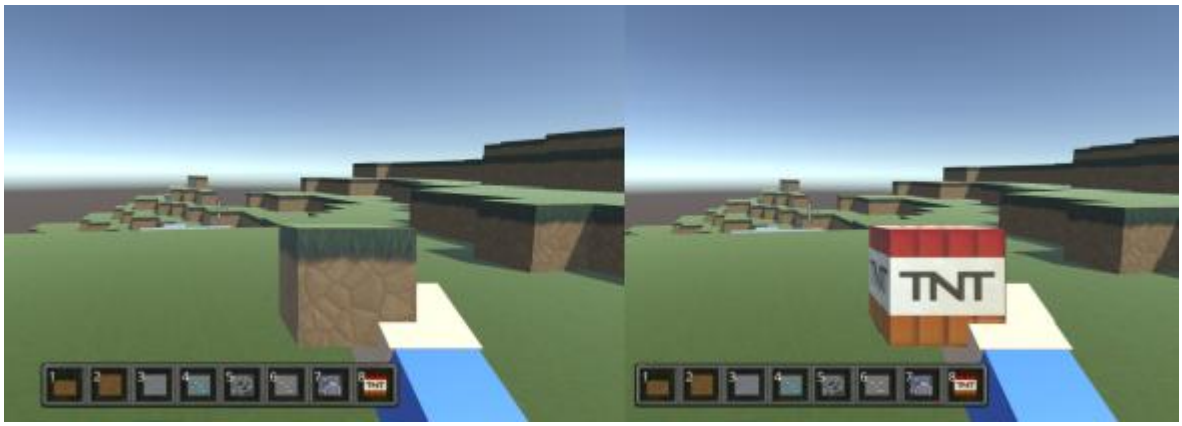


Figura 7 - Demonstração da troca do bloco selecionado

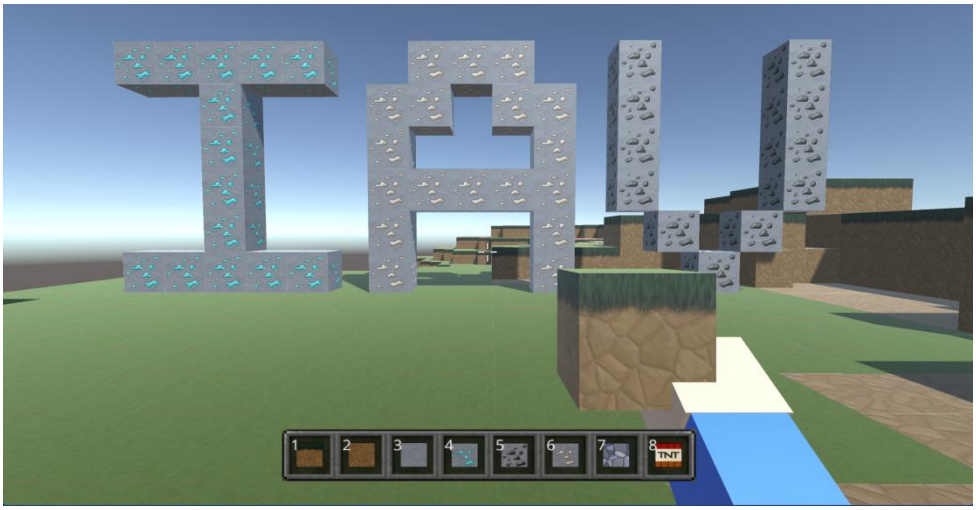


Figura 8 - Exemplo de construção por parte do jogador

Aprendizagem Automática

Para o desenvolvimento da componente de aprendizagem automática do projeto, utilizámos a biblioteca **ML-Agents da Unity**, que permite treinar agentes virtuais com base em técnicas de *machine learning*, especificamente *Reinforcement Learning* (Aprendizagem por Reforço).

Objetivo do Treino

O cenário modelado envolve a interação entre dois tipos de agentes:

- **Predadores**: cujo objetivo é **perseguir** e **tocar** a presa;
- **Presas**: cujo objetivo é **evitar serem apanhadas** pelos predadores durante o maior tempo possível.

Foram criadas duas variantes para o treino:

- 2 predadores vs 1 presa
- 3 predadores vs 1 presa

Paralelismo no Treino

Para acelerar o processo de treino, adotámos a técnica de **paralelismo**, onde múltiplas cópias do ambiente são executadas simultaneamente. Isto permitiu:

- Maior diversidade de situações por episódio;
- Aumento do número de experiências por unidade de tempo;
- Treino mais eficiente e estável dos modelos.

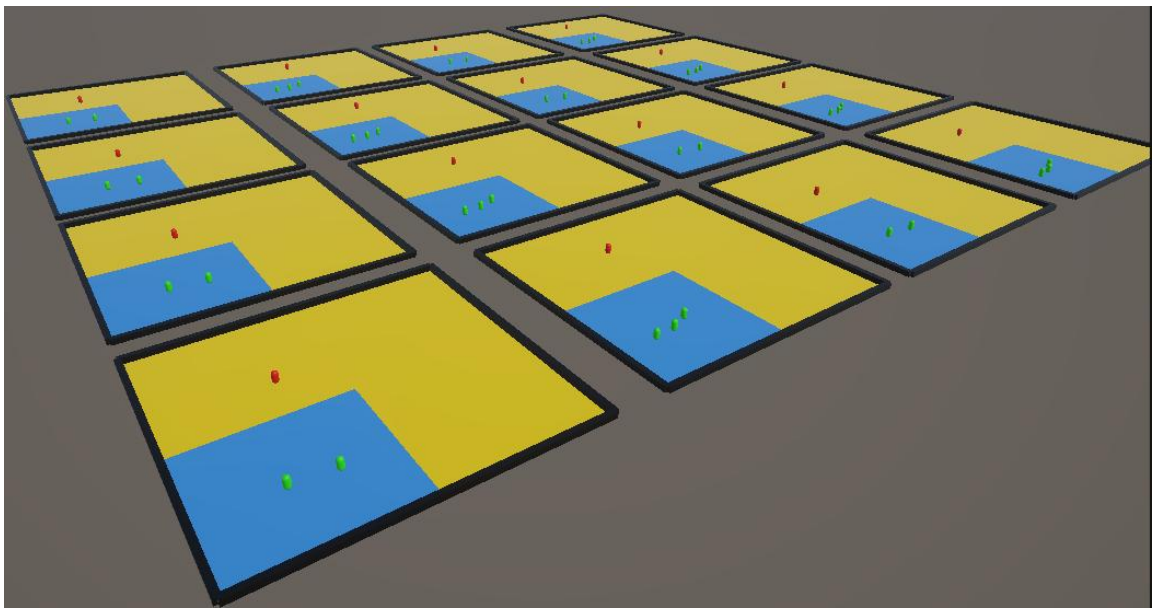


Figura 9 - Ambiente de treino dos agentes: predadores (verde) e presas (vermelho)

Algoritmo Utilizado

O algoritmo de treino usado foi o PPO (*Proximal Policy Optimization*). Ele otimiza a política do agente com segurança, e evita grandes mudanças de uma só vez, o que facilita treinos estáveis e é por isso que é amplamente utilizado em contextos de aprendizagem por reforço com ambientes contínuos, como os simulados no Unity.

Parâmetros de treino para os comportamentos: *Predator* e *Prey*

O processo de treino dos agentes foi configurado através de um ficheiro .yaml, onde foram definidos os comportamentos de cada tipo de agente: *Predator* (predador) e *Prey* (presa). Ambos os comportamentos utilizam o algoritmo PPO (*Proximal Policy Optimization*), com os mesmos parâmetros base para garantir consistência no processo de aprendizagem. Como parâmetros de treino temos:

hyperparameters

Define os hiperparâmetros do algoritmo PPO, fundamentais para o desempenho e estabilidade do treino:

- learning_rate: valor de 0.0003 utilizado como taxa de aprendizagem inicial.
- batch_size: número de experiências utilizadas por iteração de treino, definido como 64.
- buffer_size: tamanho do *buffer* de experiências (640), ou seja, o número total de passos coletados antes de atualizar a política.
- beta: coeficiente de regularização para incentivo à exploração (valor 0.005).
- epsilon: parâmetro de *clipping* PPO (0.2), limita o quanto a política pode mudar em cada atualização.
- lambd: valor lambda para estimativa de vantagem generalizada (0.95).
- num_epoch: número de épocas de treino por atualização de política (3).
- learning_rate_schedule: estratégia de agendamento da taxa de aprendizagem, definida como linear para reduzir progressivamente o valor ao longo do treino.

network_settings

- normalize: ativa a normalização das observações recebidas pelos agentes, o que facilita a convergência da política durante o treino.

reward_signals

Define o tipo de sinal de recompensa utilizado:

- extrinsic: utiliza recompensas extrínsecas atribuídas diretamente no ambiente.
    - gamma: fator de desconto temporal (0.99), prioriza recompensas futuras.
    - strength: peso da recompensa extrínseca, definido como 1.0.

self_play

Ativa o modo de treino com *Self-Play*, essencial em cenários multiagente competitivos:

- window: número de versões anteriores da política armazenadas (5).
- play_against_latest_model_ratio: probabilidade de o agente competir contra a versão mais recente do modelo (0.5).
- save_steps: intervalo de passos entre salvamentos da política atual (5000).
- swap_steps: número de passos entre trocas do oponente no self-play (10000).

max_steps

Define o número máximo de passos de treino por comportamento (200000).

summary_freq

Frequência com que os resumos de treino (estatísticas, gráficos, etc.) são registados, com um intervalo de 16000 passos.

Comportamento dos Agentes

O comportamento dos agentes foi desenvolvido com base na *framework* ML-Agents da Unity. O agente *Predator* (predador) observa continuamente a posição relativa da presa no espaço tridimensional e calcula a distância e direção até ela. Com essas informações, ele aprende a mover-se de forma eficiente em direção à presa. O movimento é controlado por duas ações contínuas (eixos X e Z), que são convertidas em deslocamento com uma velocidade fixa. Ao longo do episódio, o predador recebe uma penalização constante proporcional à duração do episódio (-1/MaxStep), incentivando-o a capturar a presa o mais rapidamente possível. Quando colide com a presa, a função `onCapture()` da presa é chamada, atribuindo uma recompensa ao predador. Todos os predadores recebem a recompensa de acordo com a sua distância à presa no momento da captura, segundo a lógica implementada no método `CalculatePredatorReward()`, ou seja, se o predador encontrava-se perto da presa (distância menor que 6) significa que ele matou ou quase matou a presa, logo recebe uma recompensa de +1, se o predador encontrava-se a uma distância menor ou igual a 12 considera-se que o predador participava na caça, logo tem uma recompensa de +0.7, se o predador estiver a uma distância maior de 12, significa que ele não participou na caça, logo é penalizado com uma recompensa de -1. O objetivo deste método é incentivar todos os predadores a participarem na caça, pois antes de utilizar-se esta lógica, apenas um dos predadores é que participava na caça da presa enquanto os restantes ficavam praticamente parados.

O predador quando colide com uma parede, recebe uma penalização (recompensa de -1) e o episódio é encerrado para todos os agentes.

O agente *Prey* (presa), por sua vez, observa a posição relativa de até três predadores. Cada predador é representado por três valores: direção normalizada (x e z) e distância até à presa. Caso haja menos de três predadores em cena, os valores correspondentes são preenchidos com zeros e uma *flag* de ausência na distância (valor impossível de distância, -1f). A presa é ligeiramente mais rápida que os predadores e é recompensada positivamente a cada passo que sobrevive (+0.01f), para incentivar estratégias de fuga prolongada. Quando é capturada, ela recebe uma penalização (recompensa de -1) e todos os agentes têm o episódio encerrado. Caso colida com uma parede, a penalização também é aplicada, promovendo a navegação segura dentro dos limites do ambiente. Decidiu-se que a penalização ao colidir com uma parede seria de -3, pois como a penalização entre os dois casos era a mesma, a presa acabava por colidir com as paredes com o intuito de fugir aos agentes, ao alterar o valor de penalização para -3, observou-se que a presa tenta evitar mais as paredes, não indo tantas vezes contra estas como no passado (quando os valores de ambas as penalizações eram os mesmos).

A estrutura apresentada de observações, recompensas e finalizações conjuntas permite um treino coordenado e competitivo entre os agentes.

Scene dos Agentes

Posteriormente à realização dos treinos dos agentes, decidiu-se criar uma *scene* simples para apresentar os resultados das duas situações apresentadas anteriormente uma presa contra dois ou três predadores. O objetivo desta *scene* é a visualização das simulações dos agentes, onde um utilizador consegue ver as simulações e utiliza as interações com o telemóvel para escolher voltar para o mundo do Minecraft ou decidir ser a presa e fugir dos predadores.

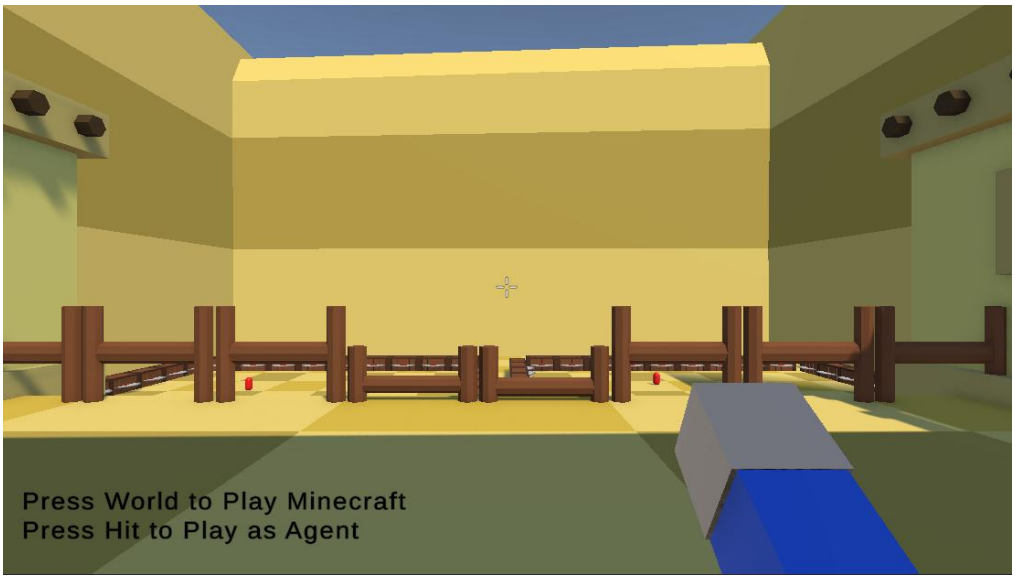


Figura 10 - Utilizador no início da scene na "varanda"



Figura 11 - Utilizador a ver as simulações dos agentes

Scene do utilizador contra os predadores

O grupo decidiu realizar uma *scene* onde os agentes (predadores), treinados anteriormente, são colocados numa arena com o utilizador. O utilizador realiza o papel da presa, onde utiliza-se o modo heurístico do agente, para o utilizador, através do telemóvel, conseguir movimentar a presa e evitar os predadores assim como as paredes. O intuito desta *scene* é o utilizador conseguir jogar contra os agentes que via nas simulações anteriores.

Esta *scene* ao ser iniciada, pergunta ao utilizador se deseja jogar contra dois ou três predadores e depois o utilizador fica num ciclo a jogar contra os agentes, quando decidir sair só é necessário utilizar os botões do telemóvel, onde um leva-o de volta para a *scene* anterior com as simulações dos agentes e outro botão faz o utilizador sair da aplicação.

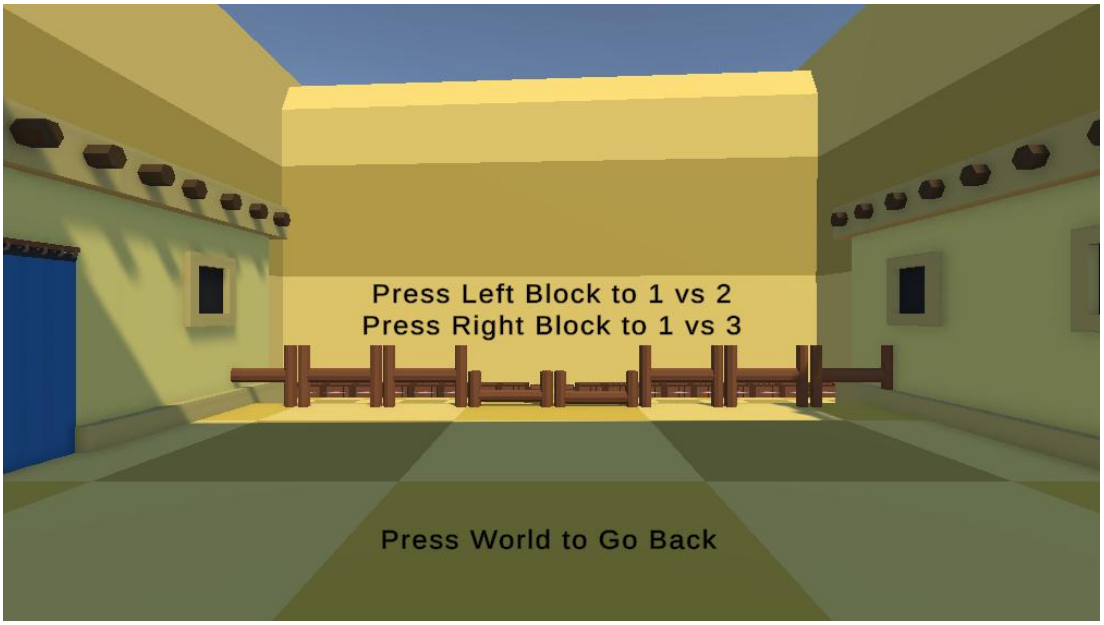


Figura 12 - Utilizador no início da scene (utilizador contra predadores)

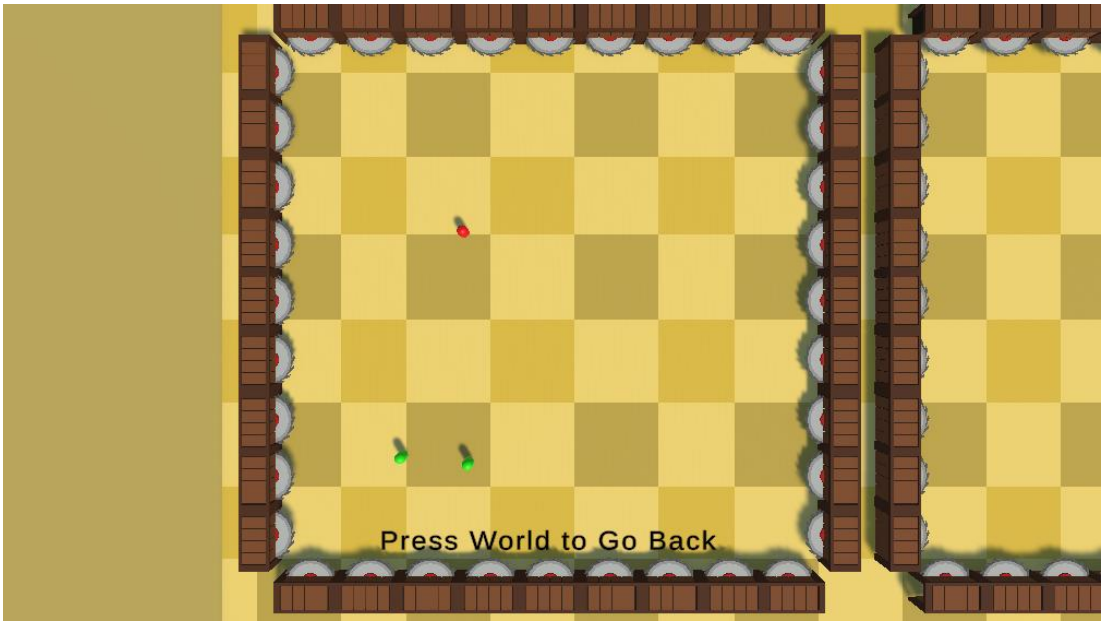


Figura 13 - Utilizador contra dois predadores



Figura 14 - Utilizador contra três predadores

Interação Pessoa-Máquina

Nesta fase do projeto, foi implementado um sistema de controlo remoto da personagem jogável através da aplicação **TouchOSC**, que permite a utilização de um dispositivo móvel como interface de controlo ao comunicar com a aplicação Unity via protocolo **OSC (*Open Sound Control*)**.

A classe principal responsável pela receção dos comandos é a `TouchOSCReceiver`, que ouve mensagens enviadas pela aplicação móvel através da porta local 7001. Cada comando OSC é associado a uma função específica através de *bindings* como /move, /see ou /buttonX, o que permite mapear gestos e toques no ecrã para ações no jogo.

Foram definidos os seguintes controlos principais:

- /move: controla o movimento da personagem (frente, trás, esquerda, direita).
- /see: controla a orientação da câmara (olhar para cima, baixo, lados).
- /button10: alterna entre *scenes* do Unity.
- /button7: alterna entre andar e correr.
- /button4: faz a personagem saltar, se estiver no chão.
- /button3 e /button5: permitem interagir com blocos, nomeadamente destruir e criar blocos.
- /button8 e /button9: permitem a escolha do bloco selecionado na *hotbar*.
- /button6: encerra o jogo.

Estes comandos são depois consultados por outras classes (`FirstPersonMovement`, `FirstPersonLook`, `Jump`), que foram modificadas para deixarem de depender do teclado ou rato, e passarem a obter os dados diretamente do `TouchOSCReceiver`. Assim, o movimento da personagem, a rotação da câmara e as ações no ambiente (como saltar ou interagir com blocos) passam a ser controladas por uma interface num dispositivo móvel.

Aqui está uma imagem que demonstra o resultado obtido.

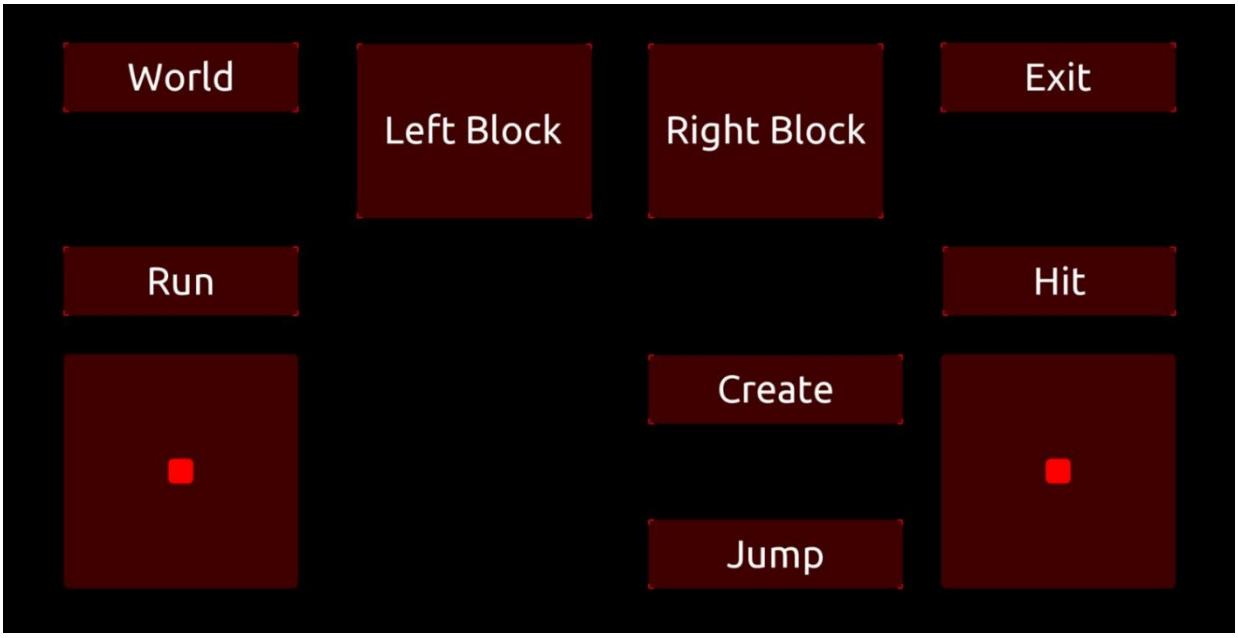


Figura 15 - Interface realizada no TouchOC

Conclusão

O desenvolvimento deste projeto permitiu integrar e aplicar, de forma prática e coesa, os conhecimentos adquiridos ao longo da unidade curricular de Interação em Ambientes Virtuais. A criação de um mundo voxel estilo Minecraft, uma area povoada por agentes inteligentes treinados com técnicas de aprendizagem por reforço, e a inclusão de uma interface móvel com TouchOSC para controlo do jogador, foram os focos principais para o desenvolvimento deste projeto.

Para trabalhos futuros, seria interessante aprofundar a integração dos agentes treinados diretamente no mundo voxel interativo, o que permitiria que os agentes atuassem no mesmo mundo que o jogador, em vez de num cenário isolado. Esta abordagem não só tornaria a simulação mais coerente, como também abriria novas possibilidades para verificar a adaptação dos agentes a ambientes dinâmicos gerados procedimentalmente, o que aproximaria o sistema ainda mais de jogos como Minecraft.

Bibliografia

Consulta:

Vídeos gravados pelo Prof. Arnaldo Abrantes - Moodle

Wikipedia Contributors. (2019, December 9). *Voxel*. Wikipedia; Wikimedia Foundation. https://en.wikipedia.org/wiki/Voxel

Texture atlas. (2022, May 30). Wikipedia. https://en.wikipedia.org/wiki/Texture_atlas

Unity ML-Agents Toolkit. (n.d.). Unity-Technologies.github.io. https://unity-technologies.github.io/ml-agents/

Manual · TouchOSC | hexler.net. (2021). Hexler.net. https://hexler.net/touchosc/manual/complete